

Project Title : Pizza Sales Analysis

Your Name : Priya Pandurang Desai

Class Name : Fortune Cloud Technologies

Subject Name : MYSQL

Date : 10-4-2026

Objective of the Project:

The main objective of this project is to analyze pizza sales data using MySQL and gain useful business insights.

This project helps to:

- Understand total orders and revenue generated
- Orders and revenue generated
- Identify popular pizza types, sizes, and categories
- Analyze customer ordering patterns (time-wise and date-wise)
- Find top-performing pizzas based on quantity and revenue
- Study sales trends and improve decision-making for business growth

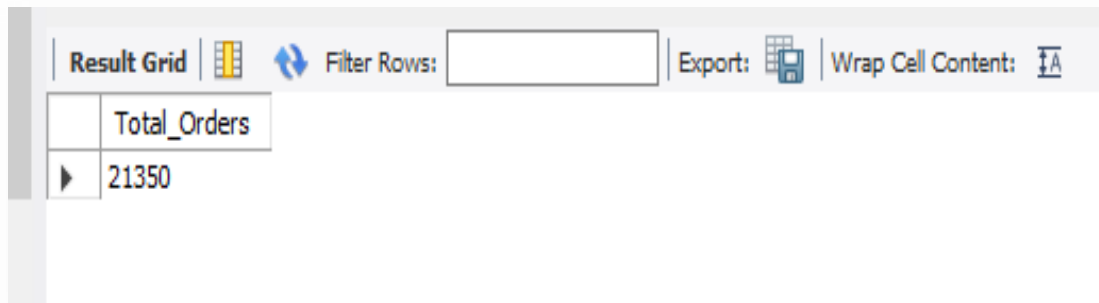


```
create database project_pizza;
```

```
use project_pizza;
```

1. Retrieve the total number of orders placed.

```
select count(*) as Total_Orders from `orders` (1);
```



The screenshot shows a database query result grid. The header row is labeled 'Total_Orders'. The first data row shows the value '21350'. The interface includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' toggle.

Total_Orders
21350

2. Calculate the total revenue generated from pizza sales.

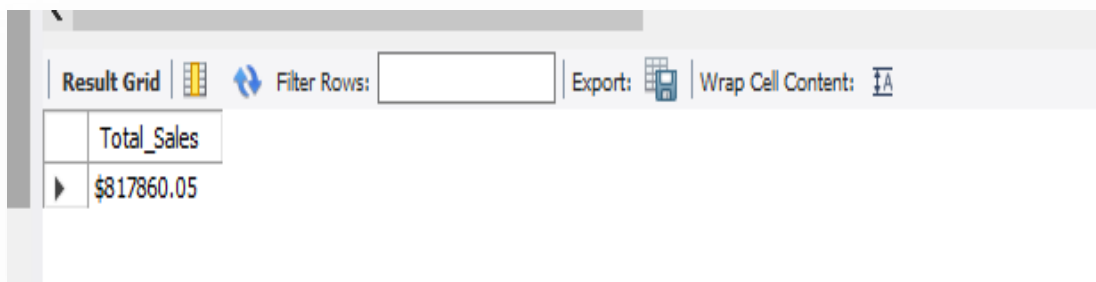
```
create view sales_info as
```

```
select od.*, (od.quantity * p.price) as Sales
```

```
from order_details od
```

```
join pizzas p on od.pizza_id=p.pizza_id;
```

```
select concat("$", round(sum(sales), 2)) as Total_Sales from sales_info;
```

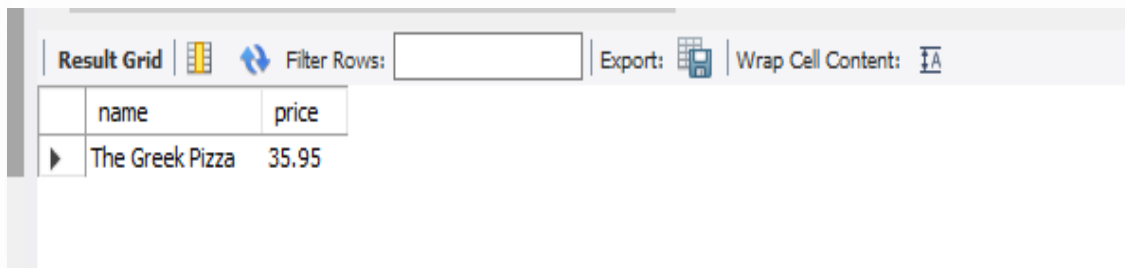


The screenshot shows a database query result grid. The header row is labeled 'Total_Sales'. The first data row shows the value '\$817860.05'. The interface includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' toggle.

Total_Sales
\$817860.05

3. Identify the highest-priced pizza.

```
select pt.name,p.price  
from pizza_types pt  
join pizzas p on pt.pizza_type_id=p.pizza_type_id  
where p.price=(select max(price) from pizzas p);
```

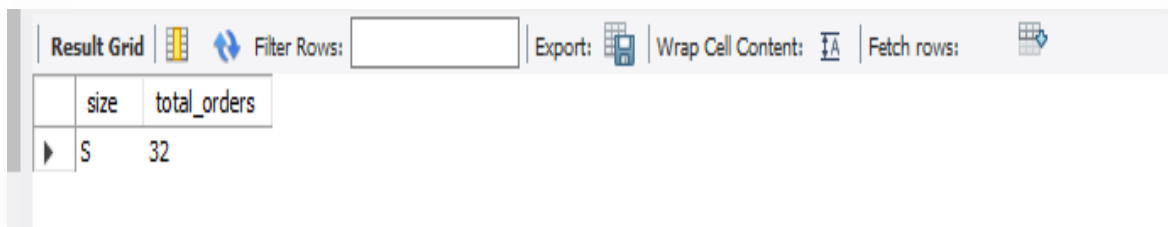


The screenshot shows a database interface with a 'Result Grid' tab. The grid has two columns: 'name' and 'price'. The first row contains 'The Greek Pizza' and '35.95'. Above the grid, there is a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox.

	name	price
▶	The Greek Pizza	35.95

4. Identify the most common pizza size ordered.

```
select size,count(pizza_id) as total_orders from pizzas group by size  
order by total_orders desc limit 1;
```

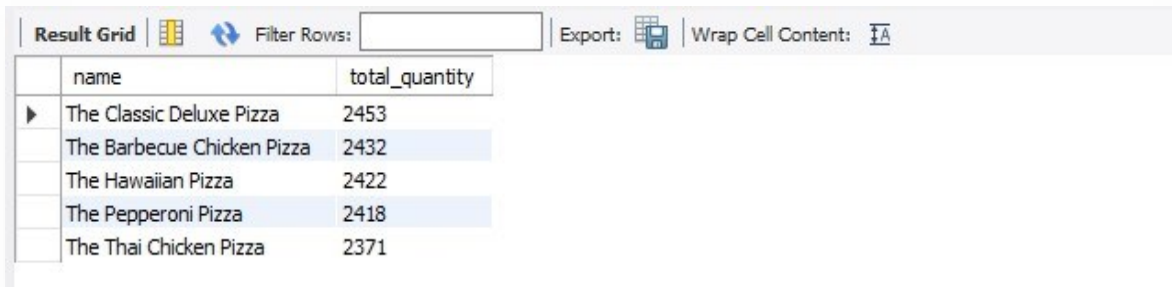


The screenshot shows a database interface with a 'Result Grid' tab. The grid has two columns: 'size' and 'total_orders'. The first row contains 'S' and '32'. Above the grid, there is a 'Filter Rows' input field, an 'Export' button, a 'Wrap Cell Content' checkbox, and a 'Fetch rows' button.

	size	total_orders
▶	S	32

5. List the top 5 most ordered pizza types along with their quantities.

```
SELECT pt.name AS pizza_name,  
SUM(od.quantity) AS total_quantity  
FROM order_details od  
JOIN pizzas p  
ON od.pizza_id = p.pizza_id  
JOIN pizza_types pt  
ON p.pizza_type_id = pt.pizza_type_id  
GROUP BY pt.name  
ORDER BY total_quantity DESC  
LIMIT 5;
```



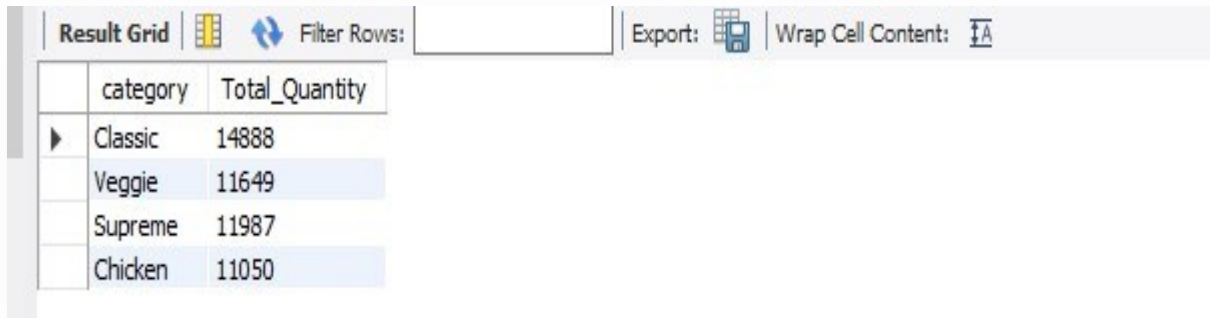
The screenshot shows a database query result grid with the following data:

	name	total_quantity
▶	The Classic Deluxe Pizza	2453
	The Barbecue Chicken Pizza	2432
	The Hawaiian Pizza	2422
	The Pepperoni Pizza	2418
	The Thai Chicken Pizza	2371

6. Join the necessary tables to find the total quantity of each pizza category ordered.

```
select t.category,sum(od.quantity) as Total_Quantity  
from pizzas p  
join pizza_types t  
on p.pizza_type_id=t.pizza_type_id
```

```
join order_details od
on od.pizza_id=p.pizza_id
group by t.category;
```



The screenshot shows a database query result grid. At the top, there is a toolbar with icons for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar is a table with two columns: 'category' and 'Total_Quantity'. The table contains four rows of data: 'Classic' with a total quantity of 14888, 'Veggie' with 11649, 'Supreme' with 11987, and 'Chicken' with 11050. The 'Classic' row is highlighted with a blue background.

category	Total_Quantity
Classic	14888
Veggie	11649
Supreme	11987
Chicken	11050

7. Determine the distribution of orders by hour of the day.

```
SELECT EXTRACT(HOUR FROM time) AS order_hour,
COUNT(order_id) AS total_orders
FROM orders
GROUP BY order_hour
ORDER BY order_hour;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_hour	Total_orders
▶	9	1
	10	8
	11	1231
	12	2520
	13	2455
	14	1472
	15	1468
	16	1920
	17	2336
	18	2399
	19	2009
	20	1642
	21	1198
	22	663
	23	28

8 Join relevant tables to find the category-wise distribution of pizzas.

```
select category,count(pizza_type_id) as Total_Pizza
from pizza_types
group by category
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:

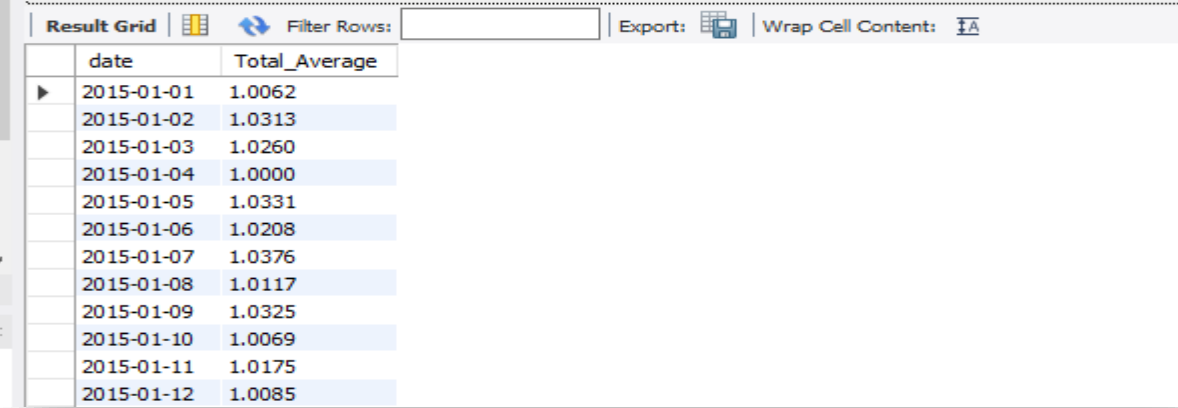


	category	Total_Pizza
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9

9. Group the orders by date and calculate the average number of pizzas ordered per day.

```
select o.date,avg(od.quantity) as Total_Average
from `orders` (1) `o`
join order_details od
```

```
on o.order_id=od.order_id  
group by o.date;
```



The screenshot shows a database query result grid. The grid has two columns: 'date' and 'Total_Average'. The data is for the month of January 2015, from the 1st to the 12th. The 'Total_Average' values are as follows:

date	Total_Average
2015-01-01	1.0062
2015-01-02	1.0313
2015-01-03	1.0260
2015-01-04	1.0000
2015-01-05	1.0331
2015-01-06	1.0208
2015-01-07	1.0376
2015-01-08	1.0117
2015-01-09	1.0325
2015-01-10	1.0069
2015-01-11	1.0175
2015-01-12	1.0085

Determine the top 3 most ordered pizza types based on revenue.

```
select t.name,sum(od.quantity * p.price) as revenue  
from pizzas p  
join pizza_types t  
on p.pizza_type_id=t.pizza_type_id  
join order_details od  
on od.pizza_id=p.pizza_id  
group by t.name  
order by revenue desc  
limit 3;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	name	revenue				
▶	The Thai Chicken Pizza	43434.25				
	The Barbecue Chicken Pizza	42768				
	The California Chicken Pizza	41409.5				

11. Calculate the percentage contribution of each pizza type to total revenue.

```

select t.name,
round(sum(od.quantity * p.price) * 100 /
(select sum(od1.quantity * p1.price)
from order_details od1
join pizzas p1
on od1.pizza_id = p1.pizza_id), 2) as percentage
from pizzas p
join pizza_types t
on p.pizza_type_id = t.pizza_type_id
join order_details od
on od.pizza_id = p.pizza_id
group by t.name;

```


Result Grid		Filter Rows:		Export:	Wrap Cell Content:
	name	percentage			
▶	The Hawaiian Pizza	3.95			
	The Classic Deluxe Pizza	4.67			
	The Five Cheese Pizza	3.19			
	The Italian Supreme Pizza	4.09			
	The Mexicana Pizza	3.27			
	The Thai Chicken Pizza	5.31			
	The Prosciutto and Arugula Pizza	2.96			
	The Barbecue Chicken Pizza	5.23			
	The Greek Pizza	3.48			
	The Spinach Supreme Pizza	1.87			
	The Green Garden Pizza	1.71			
	The Italian Capocollo Pizza	3.07			
	The Spicy Italian Pizza	4.26			
	The Spinach Pesto Pizza	1.91			
	The Vegetables + Vegetables Pi...	2.98			
	The Southwest Chicken Pizza	4.24			
	The California Chicken Pizza	5.06			
	The Pepperoni Pizza	3.69			

12. Analyze the cumulative revenue generated over time.

```

SELECT order_date,
       daily_revenue,
       SUM(daily_revenue) OVER (ORDER BY order_date) AS cumulative_revenue
FROM (
  SELECT
    o.date AS order_date,
    SUM(od.quantity * p.price) AS daily_revenue
  FROM orders o
  JOIN order_details od
    ON o.order_id = od.order_id
  JOIN pizzas p
    ON od.pizza_id = p.pizza_id
  GROUP BY o.date

```

PIZZA SALES ANALYSIS

) AS daily_sales

ORDER BY order_date;

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
	order_date	daily_revenue	cumulative_revenue
▶	2015-01-01	2713.8500000000004	2713.8500000000004
	2015-01-02	2731.8999999999996	5445.75
	2015-01-03	2662.4	8108.15
	2015-01-04	1755.4500000000003	9863.6
	2015-01-05	2065.95	11929.55
	2015-01-06	2428.95	14358.5
	2015-01-07	2202.2000000000003	16560.7
	2015-01-08	2838.3499999999995	19399.05
	2015-01-09	2127.3500000000004	21526.4
	2015-01-10	2463.95	23990.350000000002
	2015-01-11	1872.3000000000002	25862.65
	2015-01-12	1919.0500000000002	27781.7
	2015-01-13	2049.6000000000004	29831.300000000003
	2015-01-14	2527.3999999999996	32358.700000000004
	2015-01-15	1984.8000000000002	34343.500000000001
	2015-01-16	2584.45	36937.650000000001
	2015-01-17	2001.7500000000004	39001.750000000001
	2015-01-18	1976.8500000000001	40978.600000000006
	2015-01-19	2387.1499999999996	43365.750000000001
	2015-01-20	2397.9000000000005	45763.650000000001
	2015-01-21	2040.5500000000002	47804.200000000001
	2015-01-22	2496.7000000000003	50300.900000000001

Result 12 ×

13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
select t.category,sum(od.quantity * p.price) as revenue
from pizzas p
join pizza_types t
on p.pizza_type_id=t.pizza_type_id
join order_details od
on od.pizza_id=p.pizza_id
group by t.category
limit 3;
```

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	category	pizza_name	total_revenue			
▶	Chicken	The Thai Chicken Pizza	43434.25			
	Chicken	The Barbecue Chicken Pizza	42768			
	Chicken	The California Chicken Pizza	41409.5			

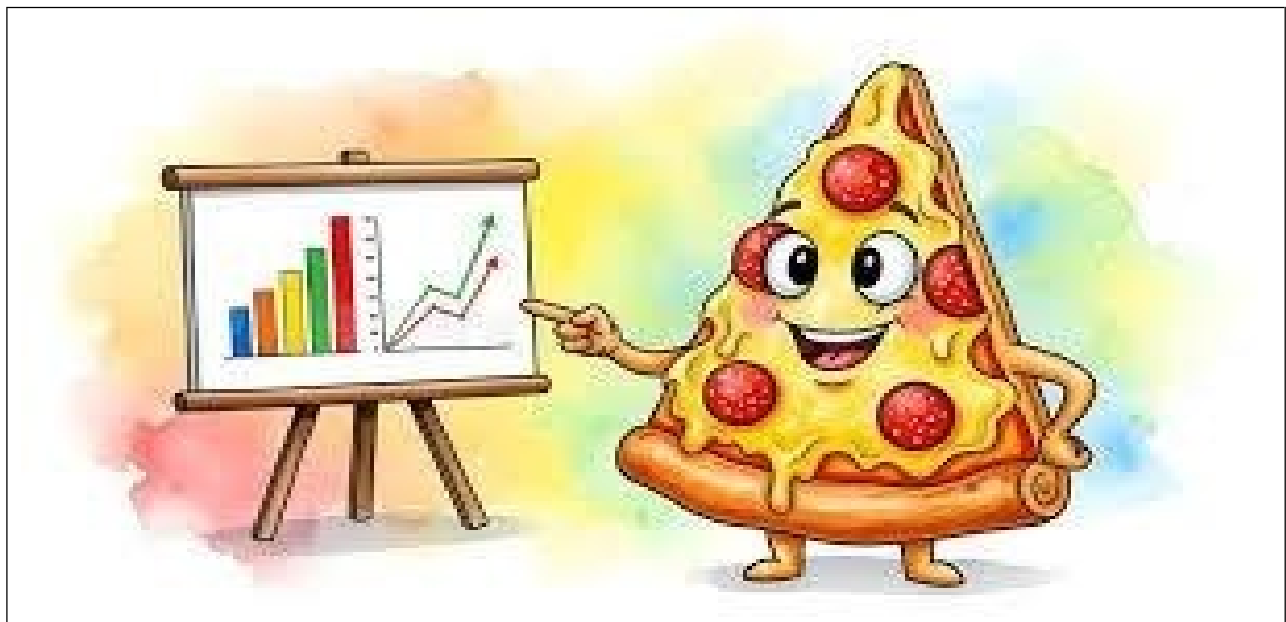
Conclusion of the Project :

From this project, we learned that data analysis helps in making smart business decisions.

Key findings include:

- Some pizza types and sizes are more popular than others
- Revenue is mainly generated from top-selling pizzas
- Customer orders vary based on time and day
- Category-wise analysis helps in understanding customer preferences

Overall, this project shows how SQL can be used to analyze real-world data and improve sales strategies, customer satisfaction, and business performance.



“Business Growth Through Data Analysis”

“Final Outcome of Pizza Sales Analysis